

Reading One Line: the Catalog

Every option at every one of the seven questions, itemized, with the example form and what it means for the note

October 2026

How to use.

This is the lookup behind the seven-question method. At each question, find the row that matches the line in front of you; the last column tells you what to carry into the note or the next question. Rows are exhaustive for the code you will meet in a review; anything not listed is rare enough to look up.

Q1. What kind of line is it?

Declarations (a new label or type comes into existence)

#	Form	Example	Carry forward
1	<code>const</code> with value	<code>const a = 1;</code>	label fixed; go to Q2 on the right side
2	<code>let</code> with value	<code>let a = 1;</code>	label may be reassigned later
3	<code>let</code> without value	<code>let a; / let a: number;</code>	holds <code>undefined</code> until assigned
4	<code>var</code>	<code>var a = 1;</code>	function-scoped, hoisted; smell
5	destructuring object	<code>const { a, b } = o;</code>	two labels <code>a</code> , <code>b</code> . With <code>o = { a: 1, b: 2 }</code> : <code>a</code> holds 1, <code>b</code> holds 2. With <code>o = { a: 1 }</code> : <code>b</code> holds <code>undefined</code> , no error. With <code>o = null</code> : THROWS
6	destructuring with rename	<code>const { a: x } = o;</code>	ONE label <code>x</code> (no label <code>a</code> exists). With <code>o = { a: 7 }</code> : <code>x</code> holds 7; <code>a</code> is not defined, so

#	Form	Example	Carry forward
24	declare	declare const g: string;	type-only; exists elsewhere

Destructuring, worked (every shape, with the input and what each label holds)

Statement	Input	Labels created	What each holds	What to check
const { id, name } = user;	user = { id: 1, name: "Ann", age: 30 }	id, name	id = 1, name = "Ann"; age is ignored	Q5: if name?: is optional, name may be undefined
const { id, name } = user;	user = { id: 1 }	id, name	id = 1, name = undefined	no error on a missing key; the crash comes at the next name.length
const { id } = user;	user = undefined	none	THROWS TypeError	Q5: can user be nothing? (find miss, optional arg)
const { a: x, b: y } = o;	o = { a: 1, b: 2 }	x, y	x = 1, y = 2; a, b do not exist as labels	reading a later is a ReferenceError
const { a = 5, b = 6 } = o;	o = { a: 1 }	a, b	a = 1, b = 6	default only fires on undefined
const { a = 5 } = o;	o = { a: null }	a	null, NOT 5	a null from JSON or a DB slips past the default
const { a: x = 5 } = o;	o = { }	x	5	rename and default together: x is the label
const { a, ...rest } = o;	o = { a: 1, b: 2, c: 3 }	a, rest	a = 1, rest = new { b: 2, c: 3 }	rest is shallow: nested objects shared with o
const { inner } = o;	o = { inner			

2. Destructuring `undefined` or `null` itself THROWS; that is the crash to look for.
3. A destructured object-valued label is an ALIAS of the original's inner object; only the top level is new when you use `...rest`.

Assignments (an existing label re-pointed, or an object mutated)

#	Form	Example	Carry forward
25	reassignment	<code>a = 2;</code>	label re-pointed; needs <code>let</code>
26	compound	<code>a += 1; , a -= 1; , a *= 2; , s += "x";</code>	reassignment with arithmetic or concat
27	increment	<code>i++; , ++i; , i--;</code>	reassignment
28	logical assignment	<code>a = b; , a &&= b; , a ??= b;</code>	reassign only if falsy / truthy / nullish
29	property set	<code>o.k = v;</code>	MUTATES <code>o</code> ; every alias sees it
30	bracket set	<code>o["k"] = v; , arr[i] = v;</code>	mutates
31	nested set	<code>o.a.b = v;</code>	mutates <code>o.a</code> , which may be shared with something else
32	delete	<code>delete o.k;</code>	mutates
33	length set	<code>arr.length = 0;</code>	mutates; empties every alias
34	destructuring assignment	<code>({ a } = o); , [x, y] = [y, x];</code>	reassigns existing labels
35	chained assignment	<code>a = b = 0;</code>	both labels point at 0

Type-only (runs nothing; note what the author claims)

#	Form	Example	Claim
58	annotation	<code>x: number</code>	<code>x</code> is a number here
59	return type	<code>): Promise<User></code>	every path returns that
60	cast	<code>x as User</code>	trust me; no check
61	non-null	<code>x!</code>	not null; no check
62	<code>satisfies</code>	<code>o satisfies Config</code>	shape check without widening
63	generic parameter	<code><T></code>	placeholder
64	<code>as const</code>	<code>[...] as const</code>	literal types
65	<code>typeof x</code> in a type	<code>let y: typeof x</code>	same type as <code>x</code>
66	<code>keyof T</code>		union of key names
67	<code>@ts-ignore</code> / <code>@ts-expect-error</code>	comment	checking off for the next line; a finding

Q2. Where does the value come from?

Literals

#	Form	Example	Q3 answer
1	string	<code>"a", 'a', `a \${x}`</code>	primitive
2	number	<code>42, 3.5, -0, 1e9, 0x1f, NaN, Infinity</code>	primitive
3	boolean	<code>true, false</code>	primitive
4	nullish	<code>null, undefined</code>	primitive

#	Form	Example	Q4 applies?
28	<code>Date.now()</code>		primitive number
29	<code>Math.*</code>	<code>Math.max(...arr)</code>	number; <code>-Infinity</code> on empty spread

Property and element reads

#	Form	Example	Can be nothing?
30	dot	<code>o.k</code>	yes if <code>k</code> is optional or absent
31	bracket, string key	<code>o["k"]</code> , <code>o[name]</code>	yes; also prototype keys on plain objects
32	index	<code>arr[i]</code> , <code>arr[0]</code> , <code>arr[arr.length - 1]</code>	yes past the end; <code>arr[-1]</code> is always <code>undefined</code>
33	<code>arr.at(i)</code>		yes; <code>-1</code> works
34	<code>arr.length</code> , <code>s.length</code>		number, never <code>undefined</code>
35	<code>map.size</code> , <code>set.size</code>		number
36	<code>map.get(k)</code>		yes when absent
37	optional chain	<code>o?.k</code> , <code>o?.[i]</code> , <code>o?.k?.j</code>	yes; by design
38	nested	<code>o.a.b.c</code>	crashes if any link is nothing
39	request fields	<code>req.params.id</code> , <code>req.query.x</code> , <code>req.body.x</code> , <code>req.headers["x"]</code>	yes; and the type is string / string[] / any
40	<code>this.field</code>		depends on how the method was called
41	<code>process.env.X</code>		<code>string</code> <code>undefined</code>

#	Form	Example	Result
64	<code>const { a } = o</code>		<code>o.a</code> ; may be <code>undefined</code>
65	<code>const [x] = arr</code>		<code>arr[0]</code>
66	<code>const { a: { b } } = o</code>		<code>o.a.b</code> ; crashes if <code>o.a</code> is nothing
67	<code>for (const [k, v] of map)</code>		each entry
68	<code>for (const [k, v] of Object.entries(o))</code>		each key-value pair

Q3. Primitive or object?

Primitives (immutable, by value, copied, no keys)

#	Type	Produced by
1	<code>string</code>	string literal, template, every string method, <code>String(x)</code> , <code>JSON.stringify</code> , <code>s.join</code> , <code>typeof</code>
2	<code>number</code>	numeric literal, arithmetic, <code>Number(x)</code> , <code>parseInt</code> , <code>arr.length</code> , <code>map.size</code> , <code>Date.now()</code> , <code>arr.indexOf</code> , <code>arr.push</code> (returns length), <code>Math.*</code>
3	<code>boolean</code>	<code>true/false</code> , comparisons, <code>===</code> , <code>!x</code> , <code>arr.includes</code> , <code>arr.some/every</code> , <code>map.has</code> , <code>Array.isArray</code> , <code>instanceof</code> , <code>in</code> , <code>delete</code> , <code>set.delete</code>
4	<code>undefined</code>	<code>let x;</code> , missing key, <code>arr[i]</code> past end, <code>map.get</code> miss, <code>find</code> miss, no <code>return</code> , <code>void</code> , <code>o?.k</code> short-circuit
5	<code>null</code>	<code>null</code> literal, DB not-found, <code>JSON</code>

#	Form	Note
7	<code>p.then(f)</code> , <code>p.catch(f)</code> , <code>p.finally(f)</code>	chaining always yields a Promise
8	<code>import("./m")</code>	dynamic import
9	<code>fs/promises</code> anything, <code>timers/promises</code> , <code>stream/promises</code>	
10	<code>util.promisify(fn)(...)</code>	
11	<code>db.query</code> , <code>pool.query</code> , <code>prisma.*</code> , <code>Model.find</code> , <code>collection.insertOne</code> , <code>redis.get</code> , <code>knex(...)</code> awaited	every DB client
12	<code>axios.get</code> , <code>got</code> , <code>ky</code> , <code>client.send(cmd)</code>	every HTTP client / SDK
13	a plain function whose body <code>return</code> s any of the above	pass-through
14	<code>arr.map(async x => ...)</code>	an ARRAY of Promises; needs <code>Promise.all</code>
15	an <code>async</code> IIFE	

Returns a value (not a ticket)

#	Form	Note
16	<code>JSON.parse</code> , <code>JSON.stringify</code>	sync
17	all array methods (<code>map</code> , <code>filter</code> , <code>find</code> , <code>sort</code> , <code>push...</code>)	sync; the callbacks may be async but the method is not
18	all string methods	sync
19	<code>Object.*</code> , <code>Math.*</code> , <code>Number(...)</code> , <code>Date.now()</code>	sync
20	<code>fs.readFileSync</code> and every <code>*Sync</code>	sync, blocking
21	<code>setTimeout</code> , <code>setInterval</code>	return an id,

#	Source	Example
8	<code>req.headers[k]</code> , <code>req.query.k</code> , <code>req.params.k</code> , <code>req.body.k</code>	not sent
9	<code>process.env.X</code>	not set
10	a function with a fall-off path	no <code>return</code> on that path
11	<code>=> { ... }</code> with no <code>return</code>	always
12	<code>let x;</code> before assignment	
13	any <code>o?.k</code>	short-circuit
14	destructuring a missing key	<code>const { a } = {}</code>
15	default param when caller passes <code>undefined</code> explicitly	default applies, so NOT nothing; but passes <code>null</code> $\rightarrow$ <code>null</code>
16	<code>Object.values(o)[0]</code>	empty object
17	<code>str.match(re)?.[1]</code>	no match / no group
18	<code>Promise.race([])</code>	never settles; rare

Yes, it can be `null`

#	Source	Example
19	explicit <code>null</code>	<code>let x = null</code>
20	DB row lookup	<code>findOne</code> $\rightarrow$ <code>null</code>
21	<code>JSON.parse("null")</code>	
22	<code>localStorage.getItem(k)</code>	miss
23	<code>regex.exec(s)</code> , <code>s.match(re)</code>	no match
24	<code>document.querySelector</code>	browser; not found
25	a <code>T null</code> return type	by declaration
26	a default param when the caller passes <code>null</code>	

#	Form	Handles <code>undefined</code> ?	Handles <code>null</code> ?	Also catches <code>0</code> / <code>""</code> / <code>false</code> ?
47	<code>"k" in x</code>	crashes on <code>undefined/null</code>		

Q6. Who else points here?

This line made a NEW address (own)

#	Form
1	object literal <code>{}</code> , <code>{ a: 1 }</code>
2	array literal <code>[]</code> , <code>[1, 2]</code>
3	<code>{ ...o }</code> , <code>{ ...a, ...b }</code> (new outer; inner objects shared)
4	<code>[...a]</code> , <code>[...a, ...b]</code> (new outer; elements shared)
5	<code>a.slice()</code> , <code>a.slice(i, j)</code> , <code>Array.from(a)</code> , <code>a.concat(b)</code>
6	<code>a.map(f)</code> , <code>a.filter(p)</code> , <code>a.flat()</code> , <code>a.flatMap(f)</code> (elements shared)
7	<code>a.toSorted()</code> , <code>a.toReversed()</code> , <code>a.toSpliced()</code> , <code>a.with(i, v)</code>
8	<code>Object.assign({}, o)</code> , <code>Object.fromEntries(...)</code> , <code>Object.keys/values/entries(o)</code>
9	<code>new Map(m)</code> , <code>new Set(s)</code> , <code>new Map()</code> , <code>new Set()</code>
10	<code>structuredClone(o)</code> (deep)
11	<code>JSON.parse(JSON.stringify(o))</code> (deep-ish)
12	<code>new C(...)</code> , <code>new Date()</code> , <code>new Error()</code>
13	a function literal
14	<code>s.split(...)</code> , <code>Object.groupBy(...)</code> , <code>Map.groupBy(...)</code>
15	

#	Form
37	<code>date.setFullYear/Month/Date/Hours/Time</code>
38	<code>Object.assign(o, src)</code> (first argument)
39	<code>Object.freeze(o)</code> , <code>Object.defineProperty(o, ...)</code>
40	<code>o.nested.k = v</code> (mutates the nested object; reaches everyone sharing <code>o.nested</code>)
41	<code>arr.forEach((o) => o.k = v)</code> (mutates each element)
42	<code>regex.exec(s)</code> with the <code>g</code> flag (advances <code>lastIndex</code>)

Who the “else” usually is

#	Holder	How to spot
43	the caller	the object came in as a parameter
44	the module / every request	a <code>const x = new Map()</code> or <code>{}</code> at module scope
45	the class / every method	<code>this.items</code>
46	the cache	<code>cache.get(k)</code> returned it
47	the original array	the object was an element from <code>find</code> / <code>[i]</code> / <code>filter</code>
48	a previous caller	the function returned the same object before
49	a listener or timer	a closure captured it
50	<code>defaults</code> / config	<code>const opts = defaults</code> instead of <code>{ ...defaults }</code>

Q7. Was the result kept?

Kept (the value goes somewhere)

#	Form
1	

#	Form	Why
21	<code>s.trim();, s.toUpperCase();, s.replace(...);, s.slice(...);</code>	new string dropped
22	<code>Object.assign({}, o);, { ...o };, [...a];</code>	copy dropped
23	<code>structuredClone(o);, JSON.parse(s);</code>	result dropped
24	<code>arr.find(p);, arr.includes(x);, map.get(k);, map.has(k);</code>	lookup dropped
25	<code>f();</code> where <code>f</code> is async	Promise dropped; rejection unhandled
26	<code>fetch(url);</code>	same
27	<code>arr.map(async x => ...);</code>	array of Promises dropped
28	<code>p.then(g);</code> with no <code>.catch</code> and not awaited	chain dropped
29	<code>x;, a + b;, o.k;, arr[0];</code>	pure expression; nothing happens
30	<code>new Date();, new User();</code>	object dropped
31	<code>(async () => { ... })();</code> with no <code>.catch</code>	Promise dropped
32	<code>const sorted = arr.sort();</code>	kept, but NOT a copy: <code>sorted === arr</code> ; the keeping misleads
33	<code>const n = arr.push(x);</code>	kept the length, not the array

The note, with every field

After Q7, write:

label holds: · <own | shared with X> · · <ticket | value>

Field	Options
value	a